Polymarket Gamma Starter Deep Operator Manual

Version 1.6.0-real - expanded guide for local research, paper ops, data/training, staged live-readiness, and guarded manual execution

Generated from the v1.6.0-real package, uploaded live configuration template, and original v1.0.1 operator manual

2026-06-14

Table of Contents

Critical safety notice

Read this first. Polymarket Gamma Starter is an operator console and workflow system. It is not financial advice, not a profit system, and not a guarantee that a thesis, prediction, signal, backtest, or workflow is correct. Prediction markets can be illiquid, volatile, misleading, and lossy. The operator is responsible for every real-money outcome.

The safest successful operation is often a blocked operation that tells you exactly why it is blocked. A blocked state is not an obstacle to bypass; it is the control system doing its job.

Core safety posture in v1.6.0-real:

- Local-first and human-in-the-loop by design.
- Live trading is disabled by default.
- Autonomous live trading is disabled by default.
- Real network trading paths are disabled by default.
- Submit and cancel paths are disabled by default.
- The live kill switch is active by default in the v1.6 package .env.example.
- Internet ingestion is disabled by default.
- Internet ingestion does not trade.
- Host training jobs are disabled by default.
- Training outputs are manual-review-only and cannot directly live-trade by default.
- Data, training, and live ledgers are runtime state and must not be packaged into release ZIPs.
- Private keys, CLOB credentials, passphrases, final confirmation phrases, user databases, session secrets, and generated ledgers must never be shared.

Required operator response to unsafe states

Situation	Required response
A page says ready but the kill switch is active	Treat live submit/cancel as blocked. Do not assume anything was sent.
A fake-local receipt appears	Treat it as local simulation only, not a Polymarket acknowledgement.
Market data is stale or missing	Do not progress live-readiness when freshness is required. Refresh or record a current snapshot.
A training signal looks attractive	Treat it as a review candidate, not a trade. It still needs research, paper review, risk gates, approvals, and live gates.
A real order status is unknown	Stop and reconcile externally and locally. Do not duplicate orders.
An internal server error appears	Capture URL, action, time, terminal log, and version. Do not continue live-adjacent workflows until resolved.
.env contains live credentials or a final phrase	Keep it local, permission-restricted, and out of ZIPs, Git, tickets, and chats.

What this manual covers

This is a new expanded operator manual for Polymarket Gamma Starter 1.6.0-real. It preserves the original v1.0.1 manual's emphasis on installation, safety, paper workflow, live-readiness, manual live submit/cancel, ledgering, reconciliation, and audit. It adds deeper coverage of the v1.2 through v1.6 milestones: Training Lab, Data Ingestion, Internet Ingestion, Host Training Jobs, Scoped Backfills, Category Datasets, and Batch Training.

Use this guide as an operating manual, not as legal, financial, tax, or investment advice. The system can help structure research and operations; it cannot make prediction markets safe.

Source basis

- Current package inspected: polymarket-gamma-starter-v1.6.0-real.zip.
- Original manual used as baseline: Polymarket_Gamma_Starter_v1.0.1_Operators_Manual.pdf.
- Uploaded live configuration template used as an environment-variable cross-check: live_config_template.env.
- Current package docs inspected: README.md, .env.example, docs/*.md, app/config.py, app/main.py, and operator modules.

v1.6.0 capability posture

Capability	v1.6.0 posture	Operator meaning
Research and evidence	Enabled local workflow	Use for thesis, counter-thesis, source collection, notes, watchlists, and evidence scoring.
Paper trading	Enabled local simulation	Local ledger only; no exchange order. Useful for process validation and risk discipline.
Market data intelligence	Local/manual by default; optional public fetch gated	Order-book metrics and freshness checks are estimates and input controls, not guarantees.
Execution-quality simulator	Deterministic local estimate	Estimates fill, spread, slippage, depth, and blockers. Not a fill guarantee.
Live readiness	Enabled as staged local records	Intent, preflight, authorization, packets, dry-run receipts, adapter requests, manual attempts.
Manual live submit/cancel	Implemented behind strict gates	Real adapter paths exist but are fail-closed unless every manual gate passes.
Autonomous trading	Dry-run/fake/manual-review posture	No autonomous live trading by default. No background scheduler starts by default.
Training Lab	Enabled local/offline lab	Datasets, features, training runs, backtests, models, and generated signal candidates.
Data ingestion	Local/import first	Network ingestion disabled unless explicitly enabled and confirmed.
Internet ingestion	Disabled by default	Public/read-only source registry with allowlists, caps, rate limits, redacted URLs, and audit.
Host training jobs	Disabled by default	Approved internal job runner only; no arbitrary shell command execution from user input.
Scoped backfill/category training	Enabled as capped workflow	Prefer scoped/category datasets to broad ingest-

Capability	v1.6.0 posture	Operator meaning
		everything jobs.

System overview and operating model

The app is best understood as a chain of explicit operator artifacts. Each stage reduces ambiguity before the next stage can be considered.

The high-level flow is:

1. Research a market and gather evidence.
2. Add notes, source packets, or a watchlist row.
3. Evaluate playbook fit and thesis quality.
4. Create a local paper trade ticket.
5. Run paper preflight and risk-budget review.
6. Approve, reject, or defer the paper ticket.
7. Execute a paper buy/sell only when the local paper gates permit.
8. Track paper positions, alerts, exit tickets, settlements, and closeout.
9. Record market-data snapshots and execution-quality simulations when needed.
10. If and only if justified, create live-readiness records: intent, preflight, authorization, packet, dry-run, adapter request.
11. Use manual execution boundary pages for blocked, fake-local, or real-live attempts.
12. Reconcile local ledgers and external account/order state.
13. Export audit records and back up runtime state.

Core principles

Local-first: Most state lives under the local data/ directory. This includes users, sessions, paper ledgers, training metadata, datasets, raw snapshots, normalized data, live-intent records, execution attempts, and audit exports.

Human-in-the-loop: Important transitions are explicit records. The app should not silently transform a vague idea into an order.

Separation of stages: Research, paper simulation, live-readiness, fake adapter rehearsal, and real manual adapter calls remain separate. Do not collapse stages to save time.

Auditability: Important records can be reviewed through UI, API, CLI, and CSV exports. If a decision matters, it should leave an audit trail.

Fail-closed execution: Live and autonomous pathways block unless the operator deliberately configures every relevant gate.

What a record proves and does not prove

Record	What it proves	What it does not prove
Watchlist row	A market was marked for	That it should be traded

Record	What it proves	What it does not prove
	review	
Evidence packet	Sources and observations were collected	That the conclusion is true
Paper ticket	A local trade thesis was drafted	That a real order exists
Preflight result	Local gates were evaluated	That market data is current forever
Approval	A human decision was recorded	That stale risk or data can be ignored
Paper position	Local simulated exposure exists	That exchange exposure exists
Live order intent	A possible live order shape was drafted	That an order was sent
Live authorization	Human authorization was recorded	That exchange validation passed
Execution packet	An unsigned package exists	That signing/submission occurred
Dry-run receipt	Local adapter-shaped validation ran	That the CLOB acknowledged anything
Adapter request	Request shape and local gates were checked	That submit/cancel is enabled
Live order ledger row	A blocked/fake/real attempt record exists	That a remote order exists unless exchange fields confirm it

Installation and first run

Recommended directory layout

Keep release ZIPs, extracted runtime folders, backups, and secrets separate. Do not run from a cluttered downloads folder that also contains credentials or generated state.

```
mkdir -p ~/polymarket-gamma
cd ~/polymarket-gamma
unzip polymarket-gamma-starter-v1.6.0-real.zip
cd polymarket-gamma-starter-v1.6.0-real
```

Python environment

Use a virtual environment. Do not package .venv into release ZIPs.

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Optional live dependency should only be installed in a deliberate live-operator runtime after reviewing current Polymarket/CLOB SDK documentation:

```
pip install -r requirements-live-optional.txt
```

Environment file

```
cp .env.example .env
chmod 600 .env
# Edit locally. Do not paste keys or passphrases into chats, issues, or docs.
```

After editing .env, start the app:

```
python run.py
# or
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

First-run checklist

1. Open /setup and create the initial admin user.
2. Use a strong unique password.
3. Log in as admin.
4. Open /administration/config and review warnings.
5. Open /health and confirm version 1.6.0 or 1.6.0-real depending on endpoint formatting.
6. Confirm safe posture: live disabled, real network disabled, submit disabled, cancel disabled, autonomous off, kill switch active, internet ingestion disabled, host training disabled.
7. Create/import research/watchlist data before expecting populated research and workflow pages.
8. If you expose the app on LAN, verify firewall and ALLOWED_HOSTS before relying on it.

Authentication, roles, and LAN deployment

The app includes local authentication. User records live in data/users.json. Session secret material lives in data/session_secret.txt. Both are runtime state and should be backed up securely, excluded from release packages, and treated as sensitive.

Role	Intended use
admin	Manage users and perform administrative actions. At least one active admin must remain.

Role	Intended use
read_only	Review/visibility workflows. Still rely on route behavior and operator discipline.

LAN deployment guidance

LAN deployment is useful when you want to operate from another device on the same network. It also increases exposure. If HOST=0.0.0.0, the app binds to all interfaces. If firewall/routing allows it, other devices may connect.

Minimum LAN controls:

- Create the admin user before exposing the service.
- Use a strong unique admin password.
- Set ALLOWED_HOSTS to fixed local IPs/hostnames when possible.
- Restrict inbound firewall rules to trusted devices/subnets.
- Use HTTPS/reverse proxy for anything beyond short local testing.
- Keep .env mode 600 or otherwise restricted.
- Keep data/ backups encrypted.
- Avoid shared public Wi-Fi or untrusted networks.

For local-only operation, use:

```
HOST=127.0.0.1
ALLOWED_HOSTS=127.0.0.1,localhost
```

Operator console navigation

The browser UI is organized around workflow stages. Use the UI for context, forms, review, and operator navigation. Use CLI/API for repeatable audit, exports, and smoke checks.

Key UI pages

Route	Purpose
/	HTML operator page
/operator	HTML operator page
/workflow	HTML operator page
/administration/config	HTML operator page
/users	HTML operator page
/opportunities	HTML operator page
/research/sources	HTML operator page
/research/evidence	HTML operator page
/playbooks	HTML operator page
/trade-tickets	HTML operator page

Route	Purpose
/preflight	HTML operator page
/approvals	HTML operator page
/execution-queue	HTML operator page
/positions	HTML operator page
/risk-budget	HTML operator page
/market-data	HTML operator page
/execution-quality	HTML operator page
/live-config	HTML operator page
/live-order-intents	HTML operator page
/live-order-intent-preflight	HTML operator page
/live-order-authorizations	HTML operator page
/live-execution-packets	HTML operator page
/live-dry-run-adapter	HTML operator page
/live-dry-run-review	HTML operator page
/live-adapter	HTML operator page
/live-adapter-requests	HTML operator page
/live-clob-adapter	HTML operator page
/live-manual-execution	HTML operator page
/live-execution-attempts	HTML operator page
/live-manual-cancel	HTML operator page
/live-trading	HTML operator page
/live-orders	HTML operator page
/live-reconciliation	HTML operator page
/strategy-signals	HTML operator page
/autonomous-trading	HTML operator page
/data	HTML operator page
/data/scopes	HTML operator page
/data/backfills	HTML operator page
/data/internet-sources	HTML operator page
/data/internet-ingestion	HTML operator page
/data/internet-workflow	HTML operator page
/training	HTML operator page
/training/category-datasets	HTML operator page
/training/host-jobs	HTML operator page

Safety badges such as LOCAL, PAPER ONLY, LIVE DISABLED, READ ONLY, FAKE ADAPTER, LIVE GUARDED, KILL SWITCH ACTIVE, AUTONOMOUS OFF, DATA INGESTION LOCAL ONLY, and HOST JOBS DISABLED are operator-critical cues.

A populated table means records exist. It does not mean execution occurred.

Empty states are normal on first run. Create upstream records before expecting downstream boards to populate.

Daily operating workflow

Use this workflow for ordinary research and paper operation sessions.

Session start

1. Start the server.
2. Log in.
3. Open /operator, /workflow, /administration/config, and /health.
4. Confirm safe posture:
   - Live disabled.
   - Real network disabled.
   - Submit disabled.
   - Cancel disabled.
   - Kill switch active.
   - Autonomous off.
   - Internet ingestion disabled unless deliberately running data collection.
   - Host training jobs disabled unless deliberately running local training.
5. Review open tickets, open positions, stale handoffs, escalations, closeout rows, and live-readiness artifacts.

Research block

1. Use /opportunities, /markets/{market_id}, source checks, and notes.
2. Collect evidence and counter-evidence.
3. Assign or evaluate playbook fit.
4. Decide whether the market deserves a paper ticket.
5. Avoid treating market score, evidence score, or model score as an automatic trade.

Paper operations block

1. Create a trade ticket.
2. Run preflight.
3. Review risk budget.
4. Approve, reject, or defer.
5. Execute paper buy/sell only when approved and unblocked.
6. Record position plan: target, stop, max-hold, invalidation, and review cadence.

7. Use exit tickets and settlements to close the loop.

Closeout block

1. Review open tickets and positions.
2. Check approvals and execution queue for stale records.
3. Record handoffs and escalations.
4. Export important CSVs.
5. Back up data/ if material work was performed.
6. Confirm no live gates were left enabled.
7. Confirm no secrets or generated state are in the source/release folder.

Research, evidence, watchlists, and playbooks

Research is the first decision layer. Its job is to decide whether a market deserves further workflow attention and what evidence would invalidate the trade.

Good research should include:

- Market question and expiry/resolution conditions.
- Current YES/NO price and implied probability.
- Liquidity, volume, spread, and depth context.
- Thesis and counter-thesis.
- Key evidence sources and source quality.
- Invalidation triggers.
- How the market could be ambiguous or resolved unexpectedly.
- Whether the opportunity fits a repeatable playbook.

Evidence workflow

1. Start from /opportunities, /markets/{market_id}, or CLI opportunity ranking.
2. Add a watchlist row for repeated review.
3. Create notes with thesis, counter-thesis, and invalidation.
4. Use /research/sources and /research/evidence for evidence packets.
5. Score evidence as a review aid.
6. Create a ticket only when the thesis and risk context justify it.

Playbooks

Playbooks help avoid one-off emotional decisions. A playbook should define:

- Market type or category.
- Entry conditions.
- Exit conditions.
- Required evidence.
- Risk limits.

- Reasons to skip.
- Failure modes.
- Post-trade review expectations.

A playbook fit is not an order. It only says the market resembles a known strategy pattern.

Paper trading lifecycle

Paper trading is not a toy in this architecture. It is a rehearsal layer for process discipline, risk review, operational handoffs, and post-trade accountability.

Entry lifecycle

1. Research market.
2. Create or attach evidence.
3. Evaluate playbook fit.
4. Create a paper trade ticket.
5. Run preflight and risk budget.
6. Approve or reject.
7. Execute paper buy only if unblocked.
8. Create position plan and alerts.
9. Review periodically.
10. Exit or settle.

Status meanings

State	Meaning	Operator action
ready	No blocking condition detected	Continue only after human review.
ready_with_warnings	Warnings exist but no hard block	Document why proceeding is acceptable.
blocked	Hard blocker exists	Do not execute. Fix upstream data/config or reject/defer.
invalid	Required source record is malformed	Repair or recreate the source record.
missing	Required source record is absent	Create the upstream record first.

Approvals do not override stale preflight or risk gates. If an approval is old, rerun preflight.

Position management

Use /positions, alerts, position plans, exit tickets, and settlement workflows. Every position should have a reason to remain open and a reason to close.

A useful position plan includes:

- Target price or profit condition.
- Stop price or invalidation condition.
- Maximum hold time.
- Review schedule.
- Liquidity/spread concern.
- Evidence update triggers.
- Settlement plan.

Risk, preflight, approvals, execution queue, and audit

Risk control in this app is a visible workflow, not a hidden calculation. You should be able to explain why a ticket was allowed, warned, or blocked.

Paper risk settings

Paper settings such as PAPER_MAX_STAKE_PER_TRADE, PAPER_MAX_MARKET_EXPOSURE, and PAPER_MAX_TOTAL_EXPOSURE control local simulation discipline. They do not protect a real exchange account unless mirrored by live settings and operator behavior.

Execution queue

The execution queue organizes paper tickets by status. Use it to find:

- Approved and ready tickets.
- Tickets needing approval.
- Tickets blocked by preflight/risk.
- Stale approvals.
- Rejected or executed items.

Do not execute from memory. Use the queue and detail pages.

Audit

Audit records are the evidence that the process occurred. Export audit CSVs after material sessions. Include audit exports in encrypted backups, not in release ZIPs.

Market data intelligence and execution-quality simulation

Market data snapshots and execution-quality simulations help avoid acting on stale, wide, or thin books.

Market data metrics

Metric	Meaning	Operator use
Best bid/ask	Top of book prices	Check reference price and

Metric	Meaning	Operator use
		spread.
Midpoint	Average of bid/ask	Approximate reference, not executable guarantee.
Spread bps	Bid/ask spread in basis points	Wide spreads increase execution risk.
Top depth	Size near top of book	Thin depth may cause warnings or blockers.
1 percent / 5 percent depth	Depth within price bands	Liquidity assessment.
Freshness	Snapshot age/status	Stale snapshots should block live readiness when required.

Execution-quality estimates

The simulator estimates fill quantity, average fill price, unfilled quantity, notional, spread, slippage, and liquidity blockers. Treat estimates as deterministic local calculations from the snapshot. They are not promises from the exchange.

Example CLI commands:

```
python -m app.cli --market-data-snapshots --json
python -m app.cli --record-market-data-snapshot --orderbook-json @orderbook.json --source manual --json
python -m app.cli --preview-execution-quality --market-id <market> --token-id <token> --side BUY --price 0.42 --size 10 --json
```

Live-readiness architecture

Live readiness is deliberately staged. Each stage exists to stop the operator from jumping from idea to order.

Live artifact chain

1. Live config readiness.
2. Live order intent.
3. Live order intent preflight.
4. Operator authorization.
5. Unsigned execution packet.
6. Dry-run adapter receipt.
7. Dry-run review.
8. Adapter request.
9. Manual execution review/control plane.
10. CLOB adapter status/verification.
11. Manual submit/cancel attempt record.

12. Live order ledger.
13. Reconciliation.

CLOB adapter boundary

The intended module boundary for real Polymarket SDK calls is app/live_clob_adapter.py. Routes, CLI commands, templates, autonomous modules, data ingestion, and training modules should not directly call trading SDK methods.

Status pages and verification probes should remain no-network/no-sign/no-submit unless the operator deliberately enters a manual live path with all gates configured.

Manual live submit SOP

Stop at the first failed gate.

1. Review current Polymarket/CLOB SDK/API documentation externally.
2. Install optional live dependency only in the operator runtime.
3. Confirm wallet, API credentials, balances, token allowances, chain ID, funder/proxy address, and signature settings externally.
4. Set exact market/token allowlists.
5. Set tiny positive risk limits.
6. Set a unique final confirmation phrase.
7. Confirm fresh market-data snapshot and execution-quality simulation.
8. Create live intent.
9. Run live intent preflight.
10. Record operator authorization.
11. Create unsigned execution packet.
12. Record dry-run receipt.
13. Create adapter request.
14. Open /live-clob-adapter and verify dependency/credential/gate status.
15. Preview blocked or fake-local first.
16. For a real-live attempt only, deliberately set live gates and restart.
17. Record adapter_mode=real_live with exact final confirmation phrase.
18. Inspect local live order ledger.
19. Inspect external Polymarket account/order status.
20. Re-enable kill switch immediately after the live window.

Manual cancel SOP

1. Confirm exchange order ID externally.
2. Confirm it appears in local ledger if available.
3. Keep submit disabled unless separately planned.
4. Enable cancel-specific gates only for a narrow window.
5. Preview cancel with order ID, operator, reason, and confirmation.

6. Record cancel only if preview passes.
7. Inspect ledger and external exchange status.
8. Record reconciliation.
9. Re-enable kill switch and disable cancel.

Emergency cancel mode is not a general bypass. It should never become a permanent operating posture.

Live order ledger and reconciliation

The live order ledger records blocked attempts, fake-local events, and real adapter results when they occur. It should store hashes, identifiers, redacted summaries, and safety flags rather than raw secrets.

Reconciliation states

State	Meaning	Action
reconciled	Local and remote views agree	Archive/export.
local_only	Local record has no remote match	Expected for blocked/fake; investigate if real attempted.
remote_only	Remote order not in local ledger	Stop and investigate external/manual activity.
status_mismatch	Local and remote statuses differ	Do not place duplicates; inspect exchange state.
reconciliation_unavailable	Credentials/dependency/ network unavailable	Use local ledger and external exchange view.

Reconciliation is a safety process. If the status is unknown, do not assume. Verify.

Strategy signals and autonomous bridge

Strategy signals are deterministic records. They are not vague LLM recommendations and they are not orders.

Supported autonomous modes include:

Mode	Default use	Meaning
off	Default	No autonomous action.
dry_run	Preview	Evaluate deterministic signals without orders/network.
paper_only	Local simulation	Paper workflow only.
fake_adapter	Local simulation	May route through fake adapter when explicitly

Mode	Default use	Meaning
		enabled.
live_guarded	Blocked by default	Should remain blocked unless a future release proves all gates.

Autonomous live trading remains disabled. A signal may become a manual-review candidate. It still requires the normal research, paper, risk, live-readiness, and manual confirmation chain.

Training and Evaluation Lab

The Training Lab is a local/offline area for datasets, feature sets, training runs, backtests, model registry records, and generated signal candidates. It is not a live-trading engine.

Main surfaces

- /training - Training overview.
- /training/datasets - Dataset registry.
- /training/features - Feature-set registry/previews.
- /training/runs - Local training runs.
- /training/backtests - Offline backtests.
- /training/models - Model registry.
- /training/signals - Generated signal candidates queued for manual review.
- /training/host-jobs - v1.5 local host training job runner.
- /training/category-datasets - v1.6 scoped/category dataset builds.

Operator rules

- Backtests are simulations, not proof.
- Model metrics are diagnostic, not authorization.
- Leakage warnings are warnings, not guarantees that leakage is absent.
- Generated signals must remain manual-review candidates.
- Training artifacts live under runtime data directories and should not be packaged into release ZIPs.
- TRAINING_ALLOW_LIVE_EXECUTION=false should remain false.

Data ingestion foundation

The Data Lab creates a local-first path from data source metadata to raw snapshots, normalized records, labels, dataset manifests, and training datasets.

Data pipeline

1. Register a data source.

2. Preview ingestion.
3. Run ingestion only when local/network gates allow it.
4. Review raw snapshot metadata.
5. Normalize records.
6. Generate or review labels.
7. Build a manifest-backed dataset.
8. Use feature sets and training runs.
9. Queue signal candidates for manual review.

Main surfaces:

- /data - Data overview.
- /data/sources - Data source registry.
- /data/ingestion - Ingestion jobs.
- /data/snapshots - Raw snapshots.
- /data/normalized - Normalized records.
- /data/labels - Labeling workbench.
- /training/dataset-builder - Dataset builder.

Data ingestion does not trade, cancel, sign, or authorize live execution.

Internet ingestion and source registry

v1.5 introduced controlled internet ingestion for public/read-only research data. v1.6 keeps this under tight caps and adds scoped/category backfill guidance.

Internet ingestion safety model

- POLYMARKET_DATA_ALLOW_INTERNET=false by default.
- Sources are registered and approved explicitly.
- Domains must be allowlisted.
- Requests are GET/read-only by design.
- Request count, timeout, rate limit, and raw response size are capped.
- Raw responses are not stored by default.
- Operator confirmation is required by default.
- Schedules are disabled by default.
- No daemon starts on app boot.
- Ingestion does not trade.

Main surfaces:

- /data/internet-sources - Approved public/read-only internet source registry.
- /data/internet-ingestion - Preview and run gated internet ingestion.
- /data/internet-workflow - Operator workflow from internet data to training.

Internet source registry rules

Register only endpoints you are allowed to access and that are public/read-only for the intended use. Do not register private, authenticated, paywalled, or trading endpoints unless you have explicit authorization and the code path is verified as read-only.

A safe registered source should record:

- Source type.
- Base URL.
- Endpoint path.
- Query parameters.
- Allowed domain.
- Timeout.
- Rate limit.
- Whether raw responses are stored.
- Expected schema or normalization path.
- Last run status.
- Warnings and blockers.

Internet-to-training workflow

1. Register internet source.
2. Validate source.
3. Preview ingestion.
4. Run ingestion only after gates pass.
5. Review raw snapshot.
6. Normalize records.
7. Generate/review labels.
8. Build a manifest-backed dataset.
9. Preview host training job.
10. Start host job only when enabled.
11. Review metrics/artifacts.
12. Register model metadata.
13. Queue generated signals for manual review.

v1.6 scoped backfill, category datasets, and batch training

v1.6 adds scoped/category workflow controls for medium-large local training. The important idea is simple: do not ingest everything. Define the slice you need, estimate it, cap it, preview it, and build a reproducible category dataset.

Why scopes matter

Broad backfills can consume too much storage, RAM, request budget, and operator time. They also create harder reproducibility and leakage problems. Scoped/category workflows reduce those risks.

Supported scope patterns include:

- Category/theme scopes such as crypto, sports, politics/elections, or daily price up/down.
- Explicit market ID lists.
- Condition ID lists.
- Event or market slug lists.
- Date-limited resolved-market scopes.
- Active-only or resolved-only filters.
- Minimum liquidity/volume filters.

v1.6 operator flow

1. Register a data scope at /data/scopes or with CLI.
2. Preview the scope and review estimated row count and warnings.
3. Link/select approved internet or local sources.
4. Preview scoped backfill: pagination, max records, max requests, storage estimate, RAM risk, deduplication plan.
5. Start only if caps and operator confirmation are acceptable.
6. Normalize and label in batches.
7. Build a category dataset using chronological, walk-forward, holdout-by-market, or holdout-by-date split.
8. Preview host training and stay within row caps.
9. Queue generated signals for manual review only.

Practical local row caps for a 16 GB RAM host

Rows	Posture	Guidance
100k	Safe small	Good for first category tests.
250k	Safe medium	v1.6 default training max row profile.
500k	Caution large	Review storage, runtime, and RAM first.
1M	High caution	Hard default cap; blocked above this unless explicitly overridden.

Deduplication

Deduplication is best-effort. It uses stable hashes and identifying fields to avoid repeated records. It reduces duplicates but should not be treated as a proof of perfect uniqueness.

Host training job runner

Host training jobs let the operator run approved internal training/backtest/feature/signal-preview tasks on the local machine.

Safety posture:

- POLYMARKET_TRAINING_HOST_JOBS_ENABLED=false by default.
- Only approved internal job types are allowed.
- No arbitrary shell commands should be accepted from user input.
- Runtime, rows, artifact bytes, and job type are capped.
- Artifacts live under runtime data directories.
- Training outputs remain manual-review-only.

Supported job types include baseline training, threshold training, momentum training, walk-forward backtest, dataset quality scan, feature build, and signal generation preview. The package .env.example currently allowlists baseline,threshold,momentum,backtest; expand only after reviewing the corresponding code path.

Main surfaces:

- /training/host-jobs
- /api/training/host-jobs
- CLI --training-host-jobs, --preview-training-host-job, --start-training-host-job, and --cancel-training-host-job.

Environment file guide (.env)

The .env file is the most important operator-control surface in this package. It decides where the server listens, whether LAN users can reach it, whether local write workflows are enabled, whether data collection may use the network, whether host training jobs may run, and whether any live-adjacent adapter path can move beyond a blocked state.

Do not treat .env as a generic settings file. Treat it as operational policy. A populated .env can contain private keys, API credentials, account identities, live-risk limits, final confirmation phrases, and local deployment details. Never commit it, upload it, paste it into chat, or include it in a release ZIP.

v1.6.0 source of truth: start from the package .env.example. The uploaded live_config_template.env is useful as an older live-variable checklist, but it was generated for

an earlier milestone. In v1.6.0, keep POLYMARKET_LIVE_KILL_SWITCH=true unless you are in a deliberate, time-boxed live or fake-adapter operation window.

How .env is loaded

- The app loads .env from the project root at startup. Restart the server after changing important gates.
- The code also has internal safe defaults. Missing live/data/training settings usually default to blocked, disabled, or local-only.
- Several variables have legacy aliases. The manual lists the aliases because older modules and older release history intentionally preserve backward compatibility.
- Status pages should redact secrets and show presence booleans or hashes rather than raw values.

Environment groups and defaults

Group	Variables in v1.6 .env.example	Operator rule
Server and LAN exposure	7	Review all values before changing from defaults.
External public API endpoints and request behavior	5	Review all values before changing from defaults.
Application posture	3	Review all values before changing from defaults.
Paper simulation risk limits	9	Review all values before changing from defaults.
Optional future integrations	2	Review all values before changing from defaults.
Legacy live-readiness gates	9	Review all values before changing from defaults.
Live adapter boundary and manual execution controls	29	Review all values before changing from defaults.
Polymarket identity and credentials	16	Review all values before changing from defaults.
Market data and execution-quality gates	8	Review all values before changing from defaults.
Autonomous bridge controls	11	Review all values before changing from defaults.
SDK mapping and verification confirmations	6	Review all values before changing from defaults.
Training and Evaluation Lab	4	Review all values before changing from defaults.
Data ingestion and dataset	4	Review all values before

Group	Variables in v1.6 .env.example	Operator rule
builder		changing from defaults.
Internet ingestion and host training jobs	14	Review all values before changing from defaults.
v1.6 scoped backfill and category training caps	9	Review all values before changing from defaults.

Detailed variable dictionary

Server and LAN exposure

- HOST (default: 0.0.0.0) - Network interface the server binds to. 127.0.0.1 is local-only. 0.0.0.0 exposes the app on all interfaces, including LAN, subject to firewall and host routing.
- PORT (default: 8000) - TCP port for the HTTP server. Default 8000 is convenient for development and LAN testing.
- APP_RELOAD (default: true) - Development reload switch. true restarts the server when source files change. For a more stable long-running operator console, use false. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- ALLOWED_HOSTS (default: *) - Allowed Host header values. * is convenient for testing, but fixed hostnames or IPs are safer for LAN deployment.
- SECURITY_HEADERS_ENABLED (default: true) - Adds common browser security headers. Leave true unless debugging a very specific local issue. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- SESSION_COOKIE_SECURE (default: false) - When true, browser sends the session cookie only over HTTPS. Use false for plain HTTP on a private LAN; use true behind HTTPS/reverse proxy. Default is fail-closed or blank unless noted.
- SESSION_COOKIE_SAMESITE (default: lax) - SameSite policy for the session cookie. lax is a reasonable default for this local console.

External public API endpoints and request behavior

- GAMMA_BASE_URL (default: https://gamma-api.polymarket.com) - Base URL for public Polymarket Gamma market/event metadata.
- CLOB_BASE_URL (default: https://clob.polymarket.com) - Base URL for public CLOB/order-book surfaces and non-live read-only helpers.
- REQUEST_TIMEOUT_SECONDS (default: 20) - Default HTTP timeout for public read operations.

- DEFAULT_LIMIT (default: 20) - Default list/result size for CLI/API views when a more specific limit is not provided.
- SOURCE_CHECK_TIMEOUT (default: 4) - Timeout for research source availability checks.

Application posture

- APP_MODE (default: read_only) - Human-readable runtime posture label surfaced in status/readiness views. It does not by itself authorize trading.
- READ_ONLY (default: true) - Broad read-only posture for core app surfaces. Keep true unless deliberately testing a local write workflow already guarded by the app. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- LIVE_TRADING_ENABLED (default: false) - Legacy broad live feature flag. In v1.6.0 this should remain false unless a deliberate manual live window is being configured through newer live gates too. Default is fail-closed or blank unless noted.

Paper simulation risk limits

- PAPER_MAX_STAKE_PER_TRADE (default: 250) - Paper-only risk guard used by local simulation. It does not control real exchange risk unless you separately mirror the policy in live gates.
- PAPER_MAX_MARKET_EXPOSURE (default: 500) - Paper-only risk guard used by local simulation. It does not control real exchange risk unless you separately mirror the policy in live gates.
- PAPER_MAX_TOTAL_EXPOSURE (default: 2500) - Paper-only risk guard used by local simulation. It does not control real exchange risk unless you separately mirror the policy in live gates.
- PAPER_MAX_OPEN_POSITIONS (default: 20) - Paper-only risk guard used by local simulation. It does not control real exchange risk unless you separately mirror the policy in live gates.
- PAPER_MIN_LIQUIDITY (default: 1000) - Paper-only risk guard used by local simulation. It does not control real exchange risk unless you separately mirror the policy in live gates.
- PAPER_MIN_VOLUME_24HR (default: 10) - Paper-only risk guard used by local simulation. It does not control real exchange risk unless you separately mirror the policy in live gates.
- PAPER_BLOCK_EXTREME_PRICES (default: true) - Paper-only risk guard used by local simulation. It does not control real exchange risk unless you separately mirror the policy in live gates. Default is enabled for this local safety/control behavior; confirm it matches your deployment.

- PAPER_MIN_PRICE (default: 0.02) - Paper-only risk guard used by local simulation. It does not control real exchange risk unless you separately mirror the policy in live gates.
- PAPER_MAX_PRICE (default: 0.98) - Paper-only risk guard used by local simulation. It does not control real exchange risk unless you separately mirror the policy in live gates.

Optional future integrations

- OPENAI_API_KEY (default: (blank)) - Placeholder for future optional integrations. Not needed for normal local paper, data, or training workflows in this package. Default is fail-closed or blank unless noted.
- NEWS_API_KEY (default: (blank)) - Placeholder for future optional integrations. Not needed for normal local paper, data, or training workflows in this package. Default is fail-closed or blank unless noted.

Legacy live-readiness gates

- LIVE_DRY_RUN_ONLY (default: true) - Legacy live-readiness field preserved for backward compatibility. Prefer newer POLYMARKET_LIVE_* fields when available; keep fail-closed by default. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- LIVE_REQUIRE_MANUAL_APPROVAL (default: true) - Legacy live-readiness field preserved for backward compatibility. Prefer newer POLYMARKET_LIVE_* fields when available; keep fail-closed by default. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- LIVE_PRETRADE_CHECKS_ENABLED (default: true) - Legacy live-readiness field preserved for backward compatibility. Prefer newer POLYMARKET_LIVE_* fields when available; keep fail-closed by default. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- LIVE_AUDIT_REQUIRED (default: true) - Legacy live-readiness field preserved for backward compatibility. Prefer newer POLYMARKET_LIVE_* fields when available; keep fail-closed by default. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- LIVE_MAX_ORDER_NOTIONAL (default: 0) - Legacy live-readiness field preserved for backward compatibility. Prefer newer POLYMARKET_LIVE_* fields when available; keep fail-closed by default. Default is fail-closed or blank unless noted.
- LIVE_MAX_MARKET_NOTIONAL (default: 0) - Legacy live-readiness field preserved for backward compatibility. Prefer newer POLYMARKET_LIVE_* fields when available; keep fail-closed by default. Default is fail-closed or blank unless noted.
- LIVE_MAX_DAILY_NOTIONAL (default: 0) - Legacy live-readiness field preserved for backward compatibility. Prefer newer POLYMARKET_LIVE_* fields when available; keep fail-closed by default. Default is fail-closed or blank unless noted.

- LIVE_MAX_OPEN_ORDERS (default: 0) - Legacy live-readiness field preserved for backward compatibility. Prefer newer POLYMARKET_LIVE_* fields when available; keep fail-closed by default. Default is fail-closed or blank unless noted.
- LIVE_ALLOWED_MARKET_IDS (default: (blank)) - Legacy live-readiness field preserved for backward compatibility. Prefer newer POLYMARKET_LIVE_* fields when available; keep fail-closed by default. Default is fail-closed or blank unless noted.

Live adapter boundary and manual execution controls

- POLYMARKET_LIVE_MODE (default: false) - Top-level live adapter posture. Must be true before real manual live submit/cancel can even be considered. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_NETWORK_READONLY (default: false) - Allows optional authenticated read-only validation paths when other real-network gates permit. It is not submit/cancel permission. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_ALLOW_REAL_NETWORK (default: false) - Hard real-network gate. If false, real CLOB submit/cancel and real smoke tests must remain blocked. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_ENABLE_SUBMIT (default: false) - Adapter-level submit request flag. Does not submit by itself; manual submit flag, kill switch, credentials, allowlists, limits, final phrase, and source records are still required. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_ENABLE_CANCEL (default: false) - Adapter-level cancel request flag. Does not cancel by itself; cancel controls and all applicable gates still apply. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_REQUIRE_MANUAL_AUTH (default: true) - Requires a saved human authorization artifact in the staged live workflow. Keep true. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- POLYMARKET_LIVE_KILL_SWITCH (default: true) - Emergency/live adapter block. In the v1.6 package default it is true. Only set false for a narrow, deliberate live or fake-adapter operation window, then immediately set true again. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- POLYMARKET_LIVE_REQUIRE_DRY_RUN_RECEIPT (default: true) - Requires a dry-run receipt before later live-adjacent adapter request/manual execution stages can proceed. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- POLYMARKET_LIVE_READONLY_TIMEOUT_SECONDS (default: 4) - Timeout used by optional read-only validation checks.

- POLYMARKET_CLOB_HOST (default: https://clob.polymarket.com) - CLOB host used by live adapter status/readiness and any future gated CLOB client construction.
- POLYMARKET_LIVE_MANUAL_SUBMIT_ENABLED (default: false) - Enables the manual submit control plane. Must remain false except during explicit testing/live windows. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_MANUAL_CANCEL_ENABLED (default: false) - Enables the manual cancel control plane. Must remain false except during explicit cancel windows. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_FAKE_ADAPTER_ENABLED (default: false) - Allows fake-local submit/cancel simulation. Fake receipts are local records only and are not exchange acknowledgements. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_REAL_ADAPTER_EXPERIMENTAL (default: false) - Experimental indicator for real adapter work. Does not replace any hard live gates. Default is fail-closed or blank unless noted.
- POLYMARKET_RUN_REAL_LIVE_TESTS (default: false) - Real-live smoke test flag. Leave false unless performing a deliberate live-runtime verification with the required confirmation string and all other gates. Default is fail-closed or blank unless noted.
- POLYMARKET_REAL_LIVE_TEST_CONFIRMATION (default: (blank)) - Explicit phrase used to confirm real-live smoke tests. Blank means no real-live smoke test consent. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_FINAL_CONFIRMATION_PHRASE (default: (blank)) - Local phrase that must be typed exactly into final manual submit/cancel attempts. Use a unique phrase per live window; never commit it. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_AUTHORIZATION_MAX_AGE_MINUTES (default: 60) - Maximum age for live order authorization records before manual execution treats them as stale.
- POLYMARKET_LIVE_DRY_RUN_MAX_AGE_MINUTES (default: 60) - Maximum age for dry-run receipts before later stages treat them as stale.
- POLYMARKET_LIVE_ADAPTER_REQUEST_MAX_AGE_MINUTES (default: 60) - Maximum age for adapter request records before manual execution treats them as stale.
- POLYMARKET_LIVE_ENABLE_AUTONOMOUS (default: false) - Autonomous live gate. Keep false. v1.6 supports dry-run/fake/manual-review workflows, not autonomous live trading. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_MAX_ORDER_NOTIONAL (default: 0) - Newer live max notional per order. Must be a small positive number for deliberate manual live windows. 0 blocks live submit. Default is fail-closed or blank unless noted.

- POLYMARKET_LIVE_MAX_DAILY_NOTIONAL (default: 0) - Newer daily notional cap for live-adjacent operations. 0 blocks live submit. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_MAX_OPEN_ORDERS (default: 0) - Maximum open live orders. 0 blocks live submit. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_MAX_POSITION_NOTIONAL (default: 0) - Maximum live position notional by position/market context. 0 is fail-closed. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_MAX_DAILY_LOSS (default: 0) - Daily loss limit placeholder. 0 is fail-closed. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_MARKET_ALLOWLIST (default: (blank)) - Comma-separated exact market IDs allowed for live/autonomous gates. Empty means no market is allowed. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_TOKEN_ALLOWLIST (default: (blank)) - Comma-separated exact token IDs allowed for live gates. Empty means no token is allowed. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_EMERGENCY_CANCEL_ENABLED (default: false) - Cancel-only emergency path flag. It is not a general bypass and still requires credentials, audit, confirmation, and real-network gates. Default is fail-closed or blank unless noted.

Polymarket identity and credentials

- POLYMARKET_CHAIN_ID (default: 137) - Target chain ID. Polygon is 137 in the template. Verify against current Polymarket/CLOB docs before live use.
- POLYMARKET_SIGNATURE_TYPE (default: (blank)) - Signature type expected by the CLOB client. Leave blank until verified with current SDK/account setup. Default is fail-closed or blank unless noted.
- POLYMARKET_FUNDER_ADDRESS (default: (blank)) - Funder/proxy address used by some CLOB account setups. Treat as sensitive account metadata. Default is fail-closed or blank unless noted.
- POLY_ADDRESS (default: (blank)) - Wallet address alias. Treat as account identity. Status pages should show only presence/redacted values. Default is fail-closed or blank unless noted.
- POLYMARKET_WALLET_ADDRESS (default: (blank)) - Wallet address alias accepted for backward compatibility. Treat as sensitive account identity. Default is fail-closed or blank unless noted.
- POLY_PRIVATE_KEY (default: (blank)) - Private signing key alias. Secret. Never paste into chat, docs, git, or release ZIPs. Default is fail-closed or blank unless noted.

- POLYMARKET_PRIVATE_KEY (default: (blank)) - Private signing key alias. Secret. Never paste into chat, docs, git, or release ZIPs. Default is fail-closed or blank unless noted.
- POLY_API_KEY (default: (blank)) - CLOB API key alias. Secret. Never commit or share. Default is fail-closed or blank unless noted.
- POLYMARKET_CLOB_API_KEY (default: (blank)) - CLOB API key alias. Secret. Never commit or share. Default is fail-closed or blank unless noted.
- CLOB_API_KEY (default: (blank)) - CLOB API key alias. Secret. Never commit or share. Default is fail-closed or blank unless noted.
- POLY_SECRET (default: (blank)) - CLOB API secret alias. Secret. Never commit or share. Default is fail-closed or blank unless noted.
- POLYMARKET_CLOB_SECRET (default: (blank)) - CLOB API secret alias. Secret. Never commit or share. Default is fail-closed or blank unless noted.
- CLOB_SECRET (default: (blank)) - CLOB API secret alias. Secret. Never commit or share. Default is fail-closed or blank unless noted.
- POLY_PASSPHRASE (default: (blank)) - CLOB passphrase alias. Secret. Never commit or share. Default is fail-closed or blank unless noted.
- POLYMARKET_CLOB_PASSPHRASE (default: (blank)) - CLOB passphrase alias. Secret. Never commit or share. Default is fail-closed or blank unless noted.
- CLOB_PASSPHRASE (default: (blank)) - CLOB passphrase alias. Secret. Never commit or share. Default is fail-closed or blank unless noted.

Market data and execution-quality gates

- POLYMARKET_MARKET_DATA_REQUIRE_FOR_LIVE (default: true) - Requires market data freshness/quality gates for live readiness when true. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- POLYMARKET_MARKET_DATA_MAX_AGE_SECONDS (default: 300) - Maximum acceptable market-data snapshot age before it is stale for live readiness.
- POLYMARKET_MARKET_DATA_MAX_SPREAD_BPS (default: 250) - Maximum bid/ask spread in basis points before warning/blocking live-readiness checks.
- POLYMARKET_MARKET_DATA_MAX_SLIPPAGE_BPS (default: 150) - Maximum estimated slippage in basis points for execution-quality gating.
- POLYMARKET_MARKET_DATA_MIN_TOP_DEPTH (default: 10) - Minimum top-of-book depth required by market-data quality gates.

- POLYMARKET_MARKET_DATA_MIN_TOTAL_DEPTH (default: 50) - Minimum total depth required by market-data quality gates.
- POLYMARKET_MARKET_DATA_PUBLIC_FETCH_ENABLED (default: false) - Optional public-fetch gate. Default false means local/manual order-book JSON snapshots first. Default is fail-closed or blank unless noted.
- POLYMARKET_MARKET_DATA_TIMEOUT_SECONDS (default: 4) - Timeout for market-data fetch/parse boundaries.

Autonomous bridge controls

- POLYMARKET_AUTONOMOUS_MAX_ORDERS_PER_RUN (default: 0) - Autonomous-run cap. 0 blocks order creation. Applies to dry-run/fake-adapter planning, not live permission. Default is fail-closed or blank unless noted.
- POLYMARKET_AUTONOMOUS_MAX_ORDERS_PER_DAY (default: 0) - Daily autonomous order cap. 0 blocks autonomous order creation. Default is fail-closed or blank unless noted.
- POLYMARKET_AUTONOMOUS_MIN_SIGNAL_CONFIDENCE (default: (blank)) - Minimum confidence threshold for deterministic strategy signals. Default is fail-closed or blank unless noted.
- POLYMARKET_AUTONOMOUS_REQUIRE_MARKET_DATA (default: true) - Requires market-data records before autonomous preview/run can proceed. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- POLYMARKET_AUTONOMOUS_REQUIRE_EXECUTION_QUALITY (default: true) - Requires execution-quality records before autonomous preview/run can proceed. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- POLYMARKET_AUTONOMOUS_REQUIRE_PAPER_APPROVAL (default: true) - Requires paper approval linkage before autonomous flow can progress. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- POLYMARKET_AUTONOMOUS_STRATEGY_ALLOWLIST (default: (blank)) - Comma-separated strategy IDs allowed for autonomous dry-run/fake-adapter flows. Default is fail-closed or blank unless noted.
- POLYMARKET_AUTONOMOUS_DRY_RUN_BY_DEFAULT (default: true) - Defaults autonomous work to dry-run. Keep true. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- POLYMARKET_AUTONOMOUS_SCHEDULER_ENABLED (default: false) - Background autonomous scheduler. Keep false unless a future reviewed release intentionally supports it. Default is fail-closed or blank unless noted.

- POLYMARKET_AUTONOMOUS_SCHEDULER_INTERVAL_SECONDS (default: 0) - Scheduler cadence if enabled. 0 means no cadence. Default is fail-closed or blank unless noted.
- POLYMARKET_AUTONOMOUS_SCHEDULER_DRY_RUN_ONLY (default: true) - Forces scheduler to dry-run-only when a scheduler exists. Keep true. Default is enabled for this local safety/control behavior; confirm it matches your deployment.

SDK mapping and verification confirmations

- POLYMARKET_LIVE_SDK_FAMILY (default: current_py_clob_client) - Declared SDK family for adapter verification reporting.
- POLYMARKET_LIVE_SDK_SUBMIT_METHOD (default: create_order+post_order) - Expected submit method mapping used for verification/reporting.
- POLYMARKET_LIVE_SDK_CANCEL_METHOD (default: cancel) - Expected cancel method mapping used for verification/reporting.
- POLYMARKET_LIVE_READONLY_REQUEST_CONFIRMATION (default: (blank)) - Optional explicit confirmation for read-only live request validation. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_SUBMIT_SMOKE_CONFIRMATION (default: (blank)) - Optional explicit confirmation for submit smoke testing. Leave blank unless deliberately testing. Default is fail-closed or blank unless noted.
- POLYMARKET_LIVE_CANCEL_SMOKE_CONFIRMATION (default: (blank)) - Optional explicit confirmation for cancel smoke testing. Leave blank unless deliberately testing. Default is fail-closed or blank unless noted.

Training and Evaluation Lab

- TRAINING_LAB_ENABLED (default: true) - Enables Training Lab pages/workflows. Training remains local/offline and manual-review-only. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- TRAINING_DEFAULT_MODEL_TYPE (default: heuristic_baseline) - Default model/strategy type for lightweight local training previews.
- TRAINING_OUTPUTS_MANUAL_REVIEW_ONLY (default: true) - Keeps training outputs in manual review posture. Keep true. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- TRAINING_ALLOW_LIVE_EXECUTION (default: false) - Hard bridge from training to live execution. Keep false. Default is fail-closed or blank unless noted.

Data ingestion and dataset builder

- POLYMARKET_DATA_ALLOW_NETWORK (default: false) - Legacy/data-lab network gate. Default false keeps ingestion local/import only. Default is fail-closed or blank unless noted.
- POLYMARKET_DATA_INGESTION_SCHEDULER_ENABLED (default: false) - Data/internet ingestion scheduler gate. Default false; no daemon starts on app boot. Default is fail-closed or blank unless noted.
- POLYMARKET_DATA_INGESTION_MAX_ROWS (default: 100000) - Maximum rows for data ingestion/dataset builder operations.
- POLYMARKET_DATA_DEFAULT_SPLIT_METHOD (default: chronological) - Default dataset split. chronological helps reduce leakage risk compared with random splits.

Internet ingestion and host training jobs

- POLYMARKET_DATA_ALLOW_INTERNET (default: false) - Internet ingestion gate. Default false. Must be true for network ingestion, but still requires source, domain, rate, size, and confirmation gates. Default is fail-closed or blank unless noted.
- POLYMARKET_DATA_ALLOWED_DOMAINS (default: (blank)) - Comma-separated domains allowed for internet ingestion. Empty means no internet domain is allowed. Default is fail-closed or blank unless noted.
- POLYMARKET_DATA_MAX_REQUESTS_PER_RUN (default: 25) - Max network requests per internet ingestion run.
- POLYMARKET_DATA_REQUEST_TIMEOUT_SECONDS (default: 10) - Per-request timeout for internet ingestion.
- POLYMARKET_DATA_RATE_LIMIT_SECONDS (default: 1) - Delay between internet ingestion requests. Avoid hammering public endpoints.
- POLYMARKET_DATA_USER_AGENT (default: PolymarketGammaStarterLocalResearch/1.0) - User-Agent string for public/read-only ingestion requests.
- POLYMARKET_DATA_STORE_RAW_RESPONSES (default: false) - Whether to store raw HTTP responses. false reduces privacy/storage risk. Default is fail-closed or blank unless noted.
- POLYMARKET_DATA_MAX_RAW_RESPONSE_BYTES (default: 1000000) - Maximum raw response bytes stored per request/response boundary.
- POLYMARKET_DATA_REQUIRE_OPERATOR_CONFIRMATION (default: true) - Requires explicit operator confirmation for gated internet ingestion actions. Keep true. Default is enabled for this local safety/control behavior; confirm it matches your deployment.

- POLYMARKET_TRAINING_HOST_JOBS_ENABLED (default: false) - Enables local host training job runner. Default false. Jobs are approved internal types, not arbitrary shell commands. Default is fail-closed or blank unless noted.
- POLYMARKET_TRAINING_MAX_RUNTIME_SECONDS (default: 300) - Max runtime for host training jobs.
- POLYMARKET_TRAINING_MAX_ROWS (default: 100000) - Max rows a host training job may process.
- POLYMARKET_TRAINING_MAX_ARTIFACT_BYTES (default: 50000000) - Max artifact size generated by host training jobs.
- POLYMARKET_TRAINING_ALLOWED_JOB_TYPES (default: baseline,threshold,momentum,backtest) - Comma-separated allowed host job types. Keep tight and reviewed.

v1.6 scoped backfill and category training caps

- POLYMARKET_DATA_MAX_BACKFILL_RECORDS (default: 500000) - v1.6 cap for scoped backfill records. Controls broad backfill risk.
- POLYMARKET_DATA_MAX_BACKFILL_REQUESTS (default: 500) - v1.6 cap for requests in a scoped backfill job.
- POLYMARKET_DATA_BACKFILL_BATCH_SIZE (default: 1000) - Batch size for backfill/normalization/training loops.
- POLYMARKET_DATA_MAX_STORAGE_MB_PER_JOB (default: 5000) - Approximate storage cap per backfill job.
- POLYMARKET_DATA_BLOCK_LARGE_BACKFILLS_BY_DEFAULT (default: true) - Blocks large backfills unless deliberately overridden. Keep true. Default is enabled for this local safety/control behavior; confirm it matches your deployment.
- POLYMARKET_TRAINING_DEFAULT_MAX_ROWS (default: 250000) - Default rows for category/batch training workflows.
- POLYMARKET_TRAINING_HARD_MAX_ROWS (default: 1000000) - Hard row cap for training jobs before blocking.
- POLYMARKET_TRAINING_BATCH_SIZE (default: 5000) - Batch size for local training operations.
- POLYMARKET_TRAINING_BLOCK_OVER_HARD_MAX_ROWS (default: true) - Blocks training over hard max rows. Keep true. Default is enabled for this local safety/control behavior; confirm it matches your deployment.

Safe .env profiles

Use these as patterns, not as a blind replacement for your local file.

Profile A - safest local-only paper/research console

```
HOST=127.0.0.1
PORT=8000
ALLOWED_HOSTS=127.0.0.1,localhost
SESSION_COOKIE_SECURE=false
APP_MODE=read_only
READ_ONLY=true
LIVE_TRADING_ENABLED=false
POLYMARKET_LIVE_MODE=false
POLYMARKET_LIVE_ALLOW_REAL_NETWORK=false
POLYMARKET_LIVE_ENABLE_SUBMIT=false
POLYMARKET_LIVE_ENABLE_CANCEL=false
POLYMARKET_LIVE_KILL_SWITCH=true
POLYMARKET_DATA_ALLOW_INTERNET=false
POLYMARKET_TRAINING_HOST_JOBS_ENABLED=false
```

Profile B - private LAN read-only console

```
HOST=0.0.0.0
PORT=8000
ALLOWED_HOSTS=localhost,127.0.0.1,192.168.1.50
SECURITY_HEADERS_ENABLED=true
SESSION_COOKIE_SECURE=false
APP_MODE=read_only
READ_ONLY=true
LIVE_TRADING_ENABLED=false
POLYMARKET_LIVE_KILL_SWITCH=true
POLYMARKET_DATA_ALLOW_INTERNET=false
```

LAN exposure means other devices can reach the app if firewall/routing allows it. Create the admin user first, use a strong password, restrict firewall access, and avoid leaving the server open on untrusted Wi-Fi.

Profile C - controlled internet ingestion lab

```
POLYMARKET_DATA_ALLOW_INTERNET=true
POLYMARKET_DATA_ALLOWED_DOMAINS=gamma-
api.polymarket.com,clob.polymarket.com
POLYMARKET_DATA_MAX_REQUESTS_PER_RUN=10
POLYMARKET_DATA_REQUEST_TIMEOUT_SECONDS=10
POLYMARKET_DATA_RATE_LIMIT_SECONDS=1
POLYMARKET_DATA_STORE_RAW_RESPONSES=false
POLYMARKET_DATA_REQUIRE_OPERATOR_CONFIRMATION=true
```

```
POLYMARKET_DATA_INGESTION_SCHEDULER_ENABLED=false
LIVE_TRADING_ENABLED=false
POLYMARKET_LIVE_KILL_SWITCH=true
```

This profile is for public/read-only data collection only. It does not trade. Keep schedules off until the registry, caps, and storage footprint have been reviewed.

Profile D - fake-local manual adapter rehearsal

```
POLYMARKET_LIVE_MODE=true
POLYMARKET_LIVE_ALLOW_REAL_NETWORK=false
POLYMARKET_LIVE_ENABLE_SUBMIT=true
POLYMARKET_LIVE_MANUAL_SUBMIT_ENABLED=true
POLYMARKET_LIVE_FAKE_ADAPTER_ENABLED=true
POLYMARKET_LIVE_KILL_SWITCH=false
POLYMARKET_LIVE_MAX_ORDER_NOTIONAL=5
POLYMARKET_LIVE_MARKET_ALLOWLIST=exact_market_id_here
POLYMARKET_LIVE_TOKEN_ALLOWLIST=exact_token_id_here
POLYMARKET_LIVE_FINAL_CONFIRMATION_PHRASE=type-a-unique-local-phrase
```

This profile is intentionally narrow and still not live trading because real network is false. It may require the kill switch to be temporarily false for the fake adapter control plane. After the rehearsal, restore POLYMARKET_LIVE_KILL_SWITCH=true, disable submit/cancel flags, and clear the final phrase.

Real manual live window reminder

A real manual live window should never be the default .env. It is a short-lived operating state requiring current Polymarket/CLOB docs, credentials, SDK dependency, exact allowlists, tiny positive risk limits, fresh source records, dry-run artifacts, final confirmation, external account verification, ledger export, and immediate restoration of fail-closed flags afterward.

CLI operator reference

The CLI is useful for audit, repeatability, exports, and smoke checks. Use --json when integrating with scripts.

Health and live safety checks

```
python -m app.cli --live-trading-status --json
python -m app.cli --live-clob-adapter-status --json
python -m app.cli --live-readiness-checklist --json
python -m app.cli --operator-runbook --json
```

Research and evidence

```
python -m app.cli --opportunities --limit 25
python -m app.cli --opportunity-engine --limit 25 --json
```

```
python -m app.cli --research <market_id> --json
python -m app.cli --collect-evidence <market_id> --json
python -m app.cli --market-notes <market_id> --json
python -m app.cli --add-note <market_id> --note-text "thesis/counter-thesis" --note-tag research
```

Paper workflow

```
python -m app.cli --trade-tickets --json
python -m app.cli --preflight --json
python -m app.cli --approvals --json
python -m app.cli --execution-queue --json
python -m app.cli --paper-positions --json
python -m app.cli --risk-budget --json
python -m app.cli --paper-ops-closeout --json
```

Market data and execution quality

```
python -m app.cli --market-data-snapshots --json
python -m app.cli --parse-market-data-snapshot-preview --orderbook-json @orderbook.json --json
python -m app.cli --record-market-data-snapshot --orderbook-json @orderbook.json --source manual --json
python -m app.cli --preview-execution-quality --market-id <market> --token-id <token> --side BUY --price 0.42 --size 10 --json
```

Live-readiness and manual boundary

```
python -m app.cli --live-config-readiness --json
python -m app.cli --live-order-intents --json
python -m app.cli --live-order-intent-preflight --json
python -m app.cli --live-order-authorizations --json
python -m app.cli --live-execution-packets --json
python -m app.cli --live-dry-run-adapter --json
python -m app.cli --live-adapter-requests --json
python -m app.cli --preview-live-submit --adapter-request-id <id> --adapter-mode blocked --operator <name> --json
python -m app.cli --preview-live-cancel --order-id <exchange_or_fake_order_id> --adapter-mode blocked --operator <name> --json
```

Data ingestion and training

```
python -m app.cli --data-status --json
python -m app.cli --data-sources --json
python -m app.cli --preview-data-ingestion --json
python -m app.cli --data-scopes --json
python -m app.cli --preview-data-backfill --scope-id <scope> --json
```

```
python -m app.cli --training-category-datasets --json
python -m app.cli --preview-training-category-dataset --scope-id <scope> --json
python -m app.cli --training-host-jobs --json
python -m app.cli --preview-training-host-job --job-type baseline_training --json
```

Export habit

Use CSV exports after any material session. Store exports with encrypted backups, not in release ZIPs.

API and route reference

The app exposes HTML pages for operators plus JSON/CSV routes for scripts, audit, and exports. Many routes require authentication depending on session state. API routes should be treated as local operator APIs, not public services.

API group counts in this build

API prefix	Route count
/api/alerts	1
/api/auth/me	1
/api/autonomous-runs	1
/api/autonomous-runs.csv	1
/api/autonomous-runs/{run_id}	1
/api/autonomous-trading/run	1
/api/autonomous-trading/run-preview	1
/api/autonomous-trading/status	1
/api/backtests	1
/api/backtests/run	1
/api/clob/book	1
/api/data/backfills	6
/api/data/backfills.csv	1
/api/data/ingestion	4
/api/data/internet-ingestion	7
/api/data/internet-sources	3
/api/data/internet-sources.csv	1
/api/data/internet-workflow	1
/api/data/labels	4
/api/data/labels.csv	1
/api/data/normalize	2

API prefix	Route count
/api/data/normalized	1
/api/data/normalized.csv	1
/api/data/scopes	3
/api/data/scopes.csv	1
/api/data/snapshots	2
/api/data/snapshots.csv	1
/api/data/sources	2
/api/data/sources.csv	1
/api/data/status	1
/api/deployment/status	1
/api/events/top	1
/api/evidence	1
/api/evidence-probabilities	1
/api/evidence-workbench	1
/api/evidence/{packet_id}	3
/api/execution-quality	1
/api/execution-quality.csv	1
/api/execution-quality/preview	1
/api/execution-quality/record	1
/api/execution-quality/{simulation_id}	1
/api/exit-tickets	1
/api/exit-tickets/{ticket_id}	4
/api/keys/status	1
/api/live/adapter	13
/api/live/clob-adapter	5
/api/live/config	3
/api/live/dry-run-adapter	2
/api/live/dry-run-adapter.csv	1
/api/live/dry-run-review	2
/api/live/dry-run-review.csv	1
/api/live/execution-attempts	2
/api/live/execution-attempts.csv	1
/api/live/execution-control	2
/api/live/execution-packets	8
/api/live/execution-packets.csv	1

API prefix	Route count
/api/live/manual-cancel	2
/api/live/manual-execution-reviews	2
/api/live/manual-execution-reviews.csv	1
/api/live/order-intents	13
/api/live/order-intents.csv	1
/api/live/orders	4
/api/live/orders.csv	1
/api/live/reconciliation	3
/api/live/reconciliation.csv	1
/api/live/trading	3
/api/maintenance/backups	3
/api/maintenance/status	1
/api/market-data/snapshots	6
/api/market-data/snapshots.csv	1
/api/markets/opportunities	1
/api/markets/top	1
/api/markets/{market_id}	19
/api/movers	1
/api/network/status	1
/api/notes	1
/api/operator-runbook	1
/api/operator/brief	1
/api/opportunities/engine	1
/api/paper/analytics	1
/api/paper/approvals	2
/api/paper/approvals.csv	1
/api/paper/audit	2
/api/paper/audit.csv	1
/api/paper/briefing	3
/api/paper/briefing.csv	1
/api/paper/buy	1
/api/paper/execution-queue	2
/api/paper/execution-queue.csv	1
/api/paper/handoffs	6
/api/paper/handoffs.csv	1

API prefix	Route count
/api/paper/ops-aging	2
/api/paper/ops-aging.csv	1
/api/paper/ops-closeout	5
/api/paper/ops-closeout.csv	1
/api/paper/ops-escalations	7
/api/paper/ops-escalations.csv	1
/api/paper/position-alerts	1
/api/paper/positions	3
/api/paper/preflight	2
/api/paper/preflight.csv	1
/api/paper/reset	1
/api/paper/review-report	2
/api/paper/review-report.csv	1
/api/paper/risk-budget	2
/api/paper/risk-budget.csv	1
/api/paper/runbook	3
/api/paper/runbook.csv	1
/api/paper/sell	1
/api/paper/settle	1
/api/paper/settlement-candidates	1
/api/paper/settlement-preview	1
/api/paper/settlements	1
/api/paper/trades	1
/api/paper/trades.csv	1
/api/playbook-decisions	1
/api/playbook-performance	1
/api/playbook-performance.csv	1
/api/playbook-performance/{playbook_id}	1
/api/playbooks	1

API usage rules

- Use GET routes for status, listing, and exports.
- Use POST routes only when you intend to create/preview/record/update a local artifact.
- Treat CSV exports as audit material.

- Never expose API routes to the public internet without a proper authentication and network boundary.
- Do not script around safety gates. If an API returns blocked, fix the reason or stop.

Data files, backups, and packaging discipline

Runtime state lives under data/. Treat it as sensitive.

Path pattern	Contents	Handling
.env	Local settings, credentials, secrets, final phrases	Never share or package. Restrict permissions.
data/users.json	Local users and password hashes	Sensitive; encrypted backup; exclude from release ZIP.
data/session_secret.txt	Session signing secret	Sensitive; exclude from release ZIP.
data/paper/*	Paper trades, positions, tickets, approvals, ops records	Operational state; backup; exclude from release ZIP.
data/live/*	Live intents, packets, receipts, adapter requests, attempts, ledger, reconciliation	Execution-adjacent; backup securely; exclude from release ZIP.
data/training/*	Datasets, features, runs, models, backtests, signals, artifacts	Runtime state; can be large; exclude from release ZIP.
data/snapshots/*	Market and raw snapshots	Generated state; exclude from release ZIP.
.venv / venv	Python environment	Do not package. Recreate from requirements.
__pycache__, .pytest_cache	Generated cache	Do not package.

Upgrade procedure

1. Stop the server.
2. Back up data/ and .env securely.
3. Extract the new release into a separate folder.
4. Copy .env carefully after reviewing new .env.example.
5. Copy or migrate data/ if appropriate.
6. Start the new server.
7. Run health checks and route smoke checks.
8. Confirm safe posture.
9. Only then resume operator workflows.

Troubleshooting and incident response

General incident procedure

1. Stop the workflow at the current step.
2. Record URL, route, time, operator action, version, and inputs.
3. Check terminal logs for Python exceptions.
4. Export or copy the relevant local record if safe.
5. Back up data/ before manual edits.
6. Use matching API endpoint to isolate template vs data issue.
7. Patch or repair.
8. Run compile/import checks.
9. Re-test the specific route.
10. Resume only if the route is stable and safety posture is clear.

Useful checks

```
python -m compileall app
python -c "from app.main import app; print(app.title, app.version)"
python -m app.cli --live-trading-status --json
python -m app.cli --live-clob-adapter-status --json
python -m app.cli --data-status --json
python -m app.cli --training-status --json
```

Likely live-submit blockers

Blocker	Remediation
Kill switch active	Leave active unless in a deliberate, time-boxed window.
Live mode false	Enable only with complete live procedure.
Real network false	Enable only with credentials, allowlists, limits, and confirmation.
Submit disabled	Set both submit flags only for deliberate manual live/fake windows.
Missing credentials	Populate only local .env; never share.
Dependency missing	Install optional live dependency only in operator runtime.
Stale authorization/dry-run/adapter request	Regenerate upstream records.
Market/token not allowlisted	Add exact IDs only.
Confirmation mismatch	Use exact configured phrase.
Max order notional zero	Set tiny positive limit only for deliberate live window.

Data/training blockers

Blocker	Remediation
Internet ingestion disabled	Keep disabled unless explicitly running public/read-only ingestion.
Domain not allowlisted	Add exact allowed domain only after review.
Too many requests	Reduce max pages/requests or scope size.
Backfill storage/RAM risk	Split by category/date/market list; lower caps.
Host jobs disabled	Enable only after reviewing allowed job types and caps.
Training over hard max rows	Downsample, split, or deliberately adjust caps after review.
Leakage warnings	Do not ignore; revise split method or labels.

Checklists

Session start checklist

- Server started from expected folder.
- Logged in as expected user.
- /health version correct.
- /administration/config reviewed.
- Live disabled.
- Real network disabled.
- Submit/cancel disabled.
- Kill switch active.
- Autonomous off.
- Internet ingestion posture understood.
- Host training job posture understood.
- Open positions, tickets, escalations, handoffs reviewed.

Paper pre-trade checklist

- Market question and resolution conditions understood.
- Thesis and counter-thesis written.
- Invalidation defined.
- Evidence packet reviewed.
- Playbook fit reviewed.
- Liquidity/spread/depth considered.
- Paper ticket created.
- Preflight and risk budget reviewed.
- Approval recorded or ticket rejected/deferred.

- Position plan documented after paper execution.

Data/training checklist

- Source authorized and appropriate for use.
- Network gates intentionally set.
- Domain allowlist exact.
- Request limits small enough.
- Raw response storage decision made.
- Scope/category defined before broad backfill.
- Deduplication plan reviewed.
- Split strategy chosen to reduce leakage risk.
- Training output remains manual-review-only.

Pre-live checklist

- Current external Polymarket/CLOB docs reviewed.
- Optional dependency installed only in live runtime.
- Credentials and wallet verified externally.
- Allowances/balances verified externally.
- Exact market/token allowlists set.
- Tiny positive limits set.
- Final confirmation phrase set.
- Market data fresh.
- Execution-quality reviewed.
- Intent/preflight/authorization/packet/dry-run/adapter request created.
- Fake or blocked preview run first.
- Kill switch disabled only for the live window.
- Kill switch restored immediately afterward.
- Local ledger and external order status reconciled.

Closeout checklist

- Kill switch active.
- Submit/cancel disabled.
- Real network disabled unless intentionally left read-only for a documented reason.
- Autonomous scheduler disabled.
- Internet/data scheduler disabled unless deliberately scheduled.
- Host training jobs stopped or accounted for.
- CSV exports saved.
- data/ backed up securely.
- No secrets in ZIP/source/logs being shared.

Manual live enablement decision record template

Use this template for a live window. Save it with your encrypted operations records.

Field	Operator entry
Date/time UTC	
Operator	
Package version	1.6.0-real
Market ID	
Token ID	
Side / price / size	
Maximum order notional	
Daily notional remaining	
External wallet/balance verified	
Current SDK docs reviewed	
Optional live dependency installed	
Market data snapshot ID	
Execution-quality simulation ID	
Intent ID	
Preflight state	
Authorization ID	
Execution packet ID	
Dry-run receipt ID	
Adapter request ID	
Kill switch disabled at	
Final confirmation phrase matched	
Live order ledger event ID	
External order ID/status	
Kill switch re-enabled at	
Post-action notes	

Glossary

Term	Definition
Adapter boundary	The isolated layer through which real CLOB SDK calls must pass.
Adapter request	A local adapter-shaped request derived from an execution packet.
Audit ledger	Local event history used for review and

Term	Definition
	evidence.
Autonomous bridge	Deterministic signal dry-run/fake-adapter workflow; not live autonomous trading by default.
Backfill	Historical data collection job, ideally scoped by category/date/market limits.
Category dataset	v1.6 scoped dataset built for a theme/category under row/storage/runtime caps.
CLOB	Central limit order book. In this project, Polymarket order interaction surfaces.
Data scope	Reusable filter describing what data a backfill/category dataset should include.
Dry-run receipt	Local no-network adapter simulation receipt. Not an exchange acknowledgement.
Execution packet	Unsigned local package of intent/preflight/authorization fields. Not an order.
Fake adapter	Local simulator for submit/cancel shaped events. Not live trading.
Final confirmation phrase	Locally configured phrase that must match operator input before manual submit/cancel record actions.
Host training job	Approved internal training/backtest/feature/signal job run on the local host under caps.
Internet source registry	Approved public/read-only source metadata for controlled ingestion.
Kill switch	Hard safety gate that blocks live submit/cancel by default.
Live order intent	Local record of intended live order shape. Not an order.
Live order ledger	Local ledger of live-adjacent order events: blocked, fake, or real.
Market allowlist	Explicit list of markets permitted for live/autonomous gates. Empty means none.
Preflight	Local check that validates order, risk, source record, and state conditions.
Reconciliation	Read-only comparison between local ledger and available remote/order status.

Term	Definition
Strategy signal	Explicit deterministic local record consumed by autonomous dry-run/fake-adapter/manual-review logic.
Token allowlist	Explicit list of Polymarket token IDs permitted for live gates. Empty means none.